W4D3

Created time	October 28, 2026 3:14 PM

Q1

Change	Answer
Add more RAM to one server	Scale up
Add four worker machines	Scale out
Replace one CPU with a faster CPU	Scale up
Add three storage nodes	Scale out

Q2.

File size = 700 MB
Block size = 128 MB

700 / 128 = 5.4687
So 6 blocks

Q3

import math

file_size = 1025 # MB
block_size = 128 # MB

num_blocks = math.ceil(file_size / block_size) # `math.ceil()` rounds up to the next whole number

print("Number of blocks:", num_blocks)

Q4. What does scaling out mean?
D. Add more machines to share the work

Q5.

A. Stores file-system metadata. →NameNode
B. Stores actual block contents. →DataNode
C. Tracks block locations. →NameNode

D. Sends heartbeats. →DataNode
E. Serves block data to clients. →DataNode

Q6.

Client asks NameNode for block locations
↓
NameNode returns block locations
↓
Client reads blocks from DataNodes
↓
Client combines the block data

Q7.

DN1 → B1, B4
DN2 → B2
DN3 → B3

Q8.

What does the NameNode mainly store?

A. File-system metadata and block-location information

Q9.

Design B creates much more metadata pressure.

Design B creates more HDFS metadata pressure because it contains 1,000,000 small files. Each file requires metadata stored in the NameNode's memory, regardless of its size. This increases NameNode RAM usage and metadata management overhead, even though both designs contain the same 10 GB of data.

Q10.

Layout A:
10 × 2 = 20 block entries

Layout B:
1,000 × 1 = 1,000 block entries

Q11. Data locality means moving the computation to where the data is stored, instead of moving the data to the computation.

Option A: Run the task where B3 is stored is better because the processing task runs on the same machine where block B3 is stored. This reduces network traffic, minimizes data transfer overhead, and generally improves processing performance.

Hadoop's data locality principle is "Move computation to the data, not data to the computation.”

Q12. Why can millions of tiny files be inefficient in HDFS?

C. Every file and block creates metadata and processing overhead

Q13.

File size = 700 MB
Replication factor = 3

700 MB × 3 = 2,100 MB

Q14.

files_mb = [100, 250, 400]
replication_factor = 3

files_mb = [100, 250, 400]
replication_factor = 3

original_total = sum(files_mb)

replicated_total = (
original_total * replication_factor
)

print("Original:", original_total, "MB")
print("Replicated:", replicated_total, "MB")

Original: 750 MB
Replicated: 2250 MB

Q15.

DataNode stops sending heartbeats
↓
NameNode marks DataNode unavailable
↓
NameNode identifies under-replicated blocks

↓
Replacement replicas are created

Q16.

Why place replicas across different racks?

B. To keep data available during a rack-level failure

Q17.

RF 2:
5 TB × 2 = 10 TB

RF 3:
5 TB × 3 = 15 TB

Q18.

- Combine existing small files into larger files to reduce NameNode metadata overhead.
- Write incoming data in larger batches so fewer small files are created, improving HDFS storage and processing efficiency.

Q19.One possible placement is:

Replica 1 → DN1, Rack A
Replica 2 → DN2, Rack A
Replica 3 → DN3, Rack B

At least one copy is outside Rack A.

A different valid placement that spreads replicas across both racks is also acceptable.

Q20.

Original data = 12 TB
Replication factor = 3
Cluster raw storage = 48 TB

Replicated data:
12 TB × 3 = 36 TB

Remaining capacity:
48 TB - 36 TB = 12 TB